

Exercise 11 — Use Private Attributes and a Constructor

SAP S/4HANA ABAP Cloud • Unit 3 • Local Classes

Goal: Replace direct public attribute changes and `set_attributes( )` with a constructor, make the instance attributes private, and make the static counter read-only.

Template: /LRN/CL_S4D400_CLS_METHODS

Solution: /LRN/CL_S4D400_CLS_CONSTRUCTOR

Your class: ZCL_9309_LOCAL_CLASS

1. What this exercise teaches

The main idea is encapsulation. The object should control how its data is created and changed. `carrier_id` and `connection_id` become `PRIVATE`, so code outside `lcl_connection` cannot directly change them. The constructor receives the values when the object is created, validates them, stores them, and increments `conn_counter`.

2. Task 1 — Copy the template (optional)

If Exercise 10 was completed in the same local class, you can continue from it. Otherwise copy /LRN/CL_S4D400_CLS_METHODS and create your own class, for example ZCL_9309_CONSTRUCTOR.

Meaning: This gives you the Exercise 10 version as the starting point for Exercise 11.

3. Task 2 — Make the instance attributes PRIVATE

Move these two attributes from PUBLIC SECTION to PRIVATE SECTION:

PRIVATE SECTION.

```
DATA carrier_id TYPE /dmo/carrier_id.  
DATA connection_id TYPE /dmo/connection_id.
```

Why? Outside code must no longer do `connection->carrier_id = ...` or `connection->connection_id = ....`. The class itself owns these values.

4. Task 3 — Replace `set_attributes( )` with a constructor

Comment out the old `set_attributes( )` definition and implementation. Then generate an instance constructor using Eclipse quick fix: place the cursor on `lcl_connection` and press `Ctrl+1` → Generate constructor.

Constructor definition required:

```
METHODS constructor  
IMPORTING
```

```

    i_carrier_id      TYPE /dmo/carrier_id
    i_connection_id   TYPE /dmo/connection_id
RAISING
    cx_abap_invalid_value.

```

Constructor implementation:

METHOD constructor.

```

IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
    RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.

```

```

me->carrier_id      = i_carrier_id.
me->connection_id   = i_connection_id.

```

```

conn_counter = conn_counter + 1.

```

ENDMETHOD.

Meaning of ME->: ME refers to the current object instance. Therefore me->carrier_id means the private carrier_id belonging to this particular object.

Meaning of RAISING: The constructor can report invalid input by raising CX_ABAP_INVALID_VALUE.

Important order: Validate first, assign second, increment the counter third. If validation fails, the counter must not increase.

5. Task 3.5 — Make conn_counter read-only

```

CLASS-DATA conn_counter TYPE i READ-ONLY.

```

Meaning: The class can change conn_counter internally, but code outside lcl_connection cannot assign a new value to it.

6. Task 4 — Create objects with NEW #(...)

Because the constructor now requires two parameters, every NEW #() expression must supply them. The object creation must also be inside TRY...CATCH because the constructor raises CX_ABAP_INVALID_VALUE.

```

TRY.
    connection = NEW #(
        i_carrier_id      = 'LH'
        i_connection_id   = '0400'
    ).

    APPEND connection TO connections.

CATCH cx_abap_invalid_value.
    out->write( `Method call failed` ).
ENDTRY.

```

Repeat the same structure for the other two instances:

```
TRY.
  connection = NEW #(
    i_carrier_id    = 'AA'
    i_connection_id = '0017'
  ).

  APPEND connection TO connections.

CATCH cx_abap_invalid_value.
  out->write( `Method call failed` ).
ENDTRY.

TRY.
  connection = NEW #(
    i_carrier_id    = 'SQ'
    i_connection_id = '0001'
  ).

  APPEND connection TO connections.

CATCH cx_abap_invalid_value.
  out->write( `Method call failed` ).
ENDTRY.
```

7. Final Global Class code

```
CLASS zcl_9309_local_class DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC.

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.

  PROTECTED SECTION.
  PRIVATE SECTION.

ENDCLASS.

CLASS zcl_9309_local_class IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.

    DATA connection TYPE REF TO lcl_connection.
    DATA connections TYPE TABLE OF REF TO lcl_connection.

    " First Instance
    TRY.
      connection = NEW #(
        i_carrier_id    = 'LH'
        i_connection_id = '0400'
      ).
    
```

```

        APPEND connection TO connections.

    CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

" Second Instance
TRY.
    connection = NEW #(
        i_carrier_id    = 'AA'
        i_connection_id = '0017'
    ).

    APPEND connection TO connections.

    CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

" Third Instance
TRY.
    connection = NEW #(
        i_carrier_id    = 'SQ'
        i_connection_id = '0001'
    ).

    APPEND connection TO connections.

    CATCH cx_abap_invalid_value.
        out->write( `Method call failed` ).
    ENDTRY.

" Output
LOOP AT connections INTO connection.
    out->write( connection->get_output( ) ).
ENDLOOP.

ENDMETHOD.

ENDCLASS.

```

8. Final local class code

```

CLASS lcl_connection DEFINITION.

    PUBLIC SECTION.

        CLASS-DATA conn_counter TYPE i READ-ONLY.

        METHODS constructor
            IMPORTING
                i_carrier_id    TYPE /dmo/carrier_id
                i_connection_id TYPE /dmo/connection_id
            RAISING
                cx_abap_invalid_value.

```

```

METHODS get_output
  RETURNING
    VALUE(r_output) TYPE string_table.

PROTECTED SECTION.

PRIVATE SECTION.

  DATA carrier_id    TYPE /dmo/carrier_id.
  DATA connection_id TYPE /dmo/connection_id.

ENDCLASS.

CLASS lcl_connection IMPLEMENTATION.

  METHOD constructor.

    IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    me->carrier_id    = i_carrier_id.
    me->connection_id = i_connection_id.

    conn_counter = conn_counter + 1.

  ENDMETHOD.

  METHOD get_output.

    APPEND |-----| TO r_output.
    APPEND |Carrier: { carrier_id }| TO r_output.
    APPEND |Connection: { connection_id }| TO r_output.

  ENDMETHOD.

ENDCLASS.

```

9. Activate and run

1. Press Ctrl+F3 to activate.
2. Press F9 to run the class as a console application.
3. Confirm that the three connections are printed.

```

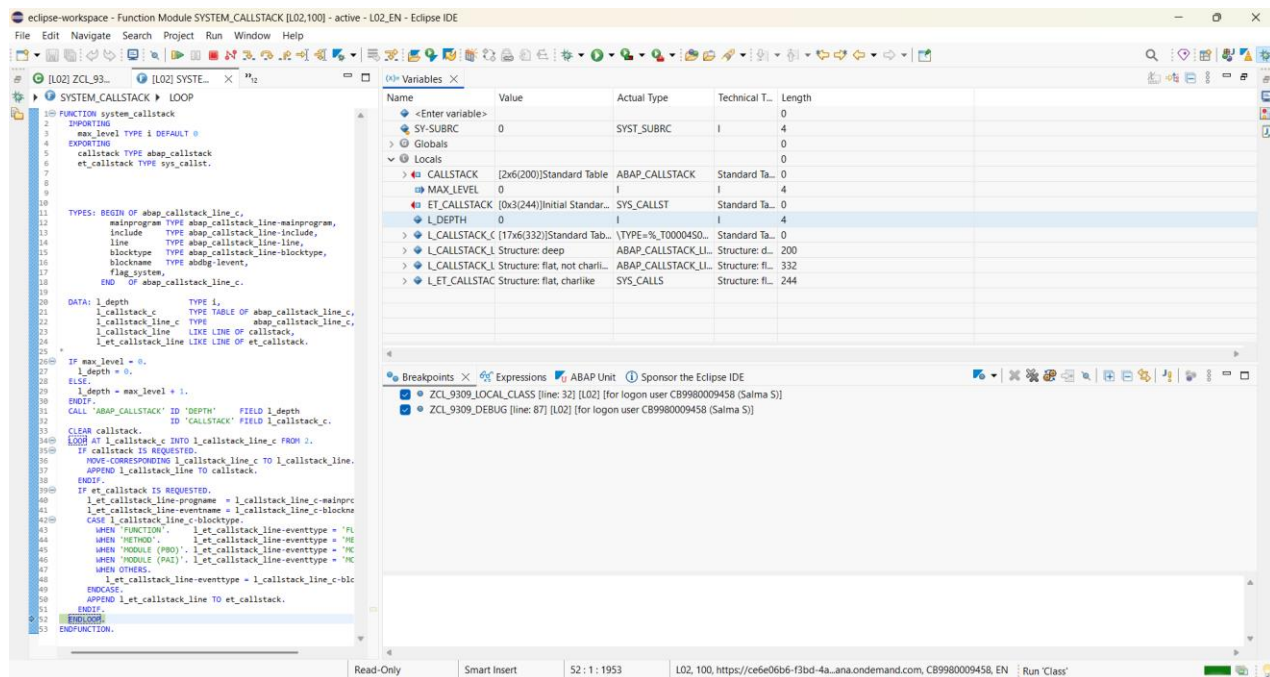
-----
Carrier: LH
Connection: 0400
-----
Carrier: AA
Connection: 0017
-----
Carrier: SQ

```

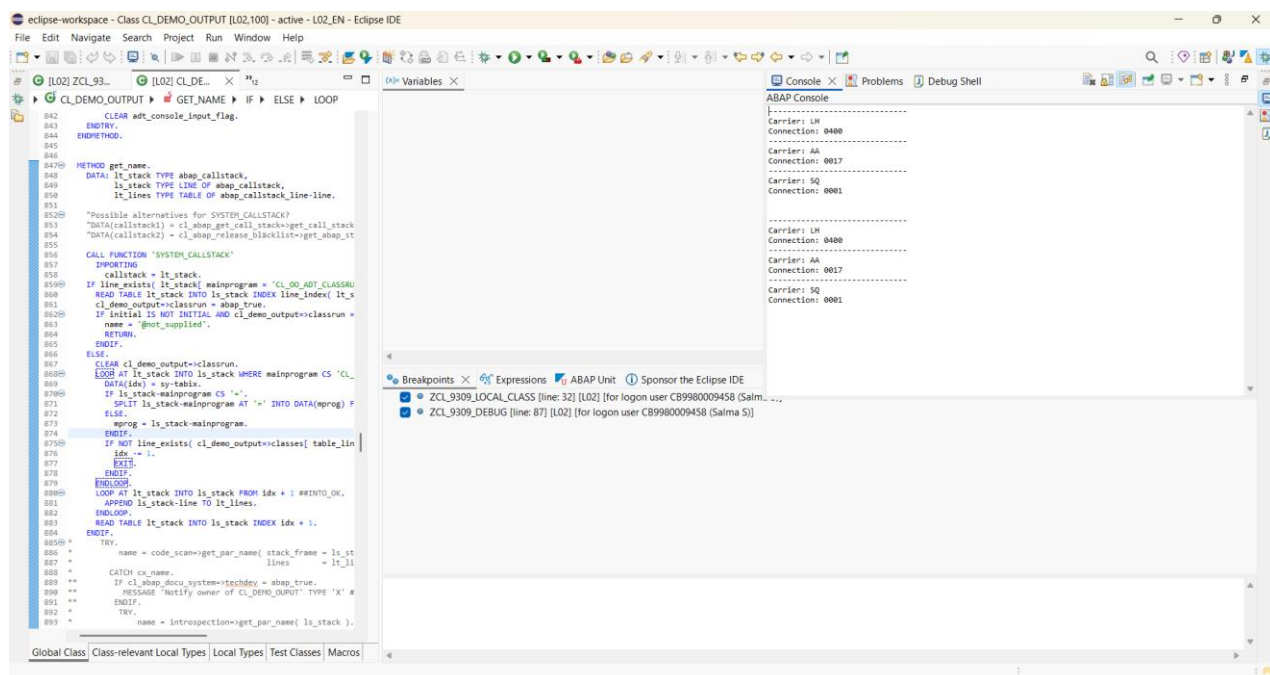
Connection: 0001

10. Debugging — what happened in your session

During debugging, the debugger entered SAP internal code such as `SYSTEM_CALLSTACK` and `CL_DEMO_OUTPUT`. That is not an Exercise 11 error. It happened because the debugger stepped into framework/system implementation.



Screenshot 1 — Debugger entered SAP's internal `SYSTEM_CALLSTACK` function.



Screenshot 2 — Debugger entered SAP's internal CL_DEMO_OUTPUT code while the console output was already produced.

Correct debugger keys: F5 = Step Into, F6 = Step Over, F7 = Step Return, F8 = Resume. For this exercise, F6/F8 are safer when you do not want to enter SAP framework code.

11. Concept summary

Concept	What it does	Why Exercise 11 uses it
PRIVATE attribute	Only the class can directly access the data.	Prevents external code from changing carrier_id and connection_id.
Constructor	Runs automatically when NEW creates an object.	Forces required values to be supplied at object creation.
RAISING	Declares an exception a method/constructor can raise.	Rejects empty carrier or connection values.
READ-ONLY	Allows reading but prevents outside assignment.	Protects conn_counter from external changes.
ME->	Refers to the current object instance.	Assigns constructor input to private instance attributes.
TRY...CATCH	Handles a raised exception.	Prevents invalid object creation from terminating the program.

12. Exercise 11 checklist

- carrier_id is PRIVATE
- connection_id is PRIVATE
- set_attributes() is commented out / no longer used
- constructor has i_carrier_id and i_connection_id
- constructor raises CX_ABAP_INVALID_VALUE for initial input
- attributes are assigned through ME->
- conn_counter is incremented only after successful validation
- conn_counter is READ-ONLY
- all NEW #(...) expressions pass constructor parameters
- all constructor calls are protected by TRY...CATCH
- class activates successfully
- F9 produces the three expected connections

Exercise 11 completed — Private Attributes and Constructor